Course Title: Cryptography and Network Security Lab	**Course Code**: BCSE309P
Faculty: Prof. Karthika V	Slot: *L15+L16*
Regno: *23BCE1099*	Name: *Arkita Barua*

LAB2 – Ex1b

PLAYFAIR CIPHER

Code

Sender

```
// Client.java - sender
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class Sender {
    private char[][] matrix;

    public Sender(String key) {
        buildMatrix(key.toUpperCase().replace('J', 'I'));
    }

    private void buildMatrix(String key) {
        boolean[] used = new boolean[26];
        matrix = new char[5][5];
        int idx = 0;

        for (char c : key.toCharArray()) {
            if (c >= 'A' && c <= 'Z' && !used[c - 'A']) {
                used[c - 'A'] = true;
                matrix[idx / 5][idx % 5] = c;
                idx++;
            }
        }

        for (char c = 'A'; c <= 'Z'; c++) {
            if (c == 'J') continue;
            if (!used[c - 'A']) {
                matrix[idx / 5][idx % 5] = c;
                idx++;
            }
```

```java
        }
    }

    private String preprocess(String text) {
        StringBuilder clean = new StringBuilder();
        for (char c : text.toUpperCase().toCharArray()) {
            if (c >= 'A' && c <= 'Z') {
                clean.append(c == 'J' ? 'I' : c);
            }
        }

        StringBuilder result = new StringBuilder();
        int i = 0;
        while (i < clean.length()) {
            char a = clean.charAt(i);
            if (i == clean.length() - 1) {
                // Last character, odd length
                result.append(a).append('X');
                break;
            }
            char b = clean.charAt(i + 1);
            if (a == b) {
                // Same letter: use 'X' as filler, and do NOT consume b yet
                result.append(a).append('X');
                i++; // move only by 1, so b is reused as first letter next
time
            } else {
                result.append(a).append(b);
                i += 2;
            }
        }
        return result.toString();
    }
    private String insertFillers(String s) {
        StringBuilder res = new StringBuilder();
        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            res.append(c);
            if (i + 1 < s.length() && s.charAt(i + 1) == c) {
                res.append('X');
            }
        }
        if (res.length() % 2 == 1) {
            res.append('X');
        }
        return res.toString();
    }
```

```java
    private int[] find(char c) {
        for (int i = 0; i < 5; i++) {
            for (int j = 0; j < 5; j++) {
                if (matrix[i][j] == c) {
                    return new int[]{i, j};
                }
            }
        }
        return new int[]{-1, -1};
    }

    public String encrypt(String plain) {
        String clean = preprocess(plain);
        StringBuilder cipher = new StringBuilder();

        for (int i = 0; i < clean.length(); i += 2) {
            char a = clean.charAt(i);
            char b = clean.charAt(i + 1);
            int[] posA = find(a);
            int[] posB = find(b);

            if (posA[0] == posB[0]) {
                cipher.append(matrix[posA[0]][(posA[1] + 1) % 5])
                      .append(matrix[posB[0]][(posB[1] + 1) % 5]);
            } else if (posA[1] == posB[1]) {
                cipher.append(matrix[(posA[0] + 1) % 5][posA[1]])
                      .append(matrix[(posB[0] + 1) % 5][posB[1]]);
            } else {
                cipher.append(matrix[posA[0]][posB[1]])
                      .append(matrix[posB[0]][posA[1]]);
            }
        }
        return cipher.toString();
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter key: ");
        String key = sc.nextLine().trim();

        System.out.print("Enter plaintext: ");
        String plaintext = sc.nextLine().trim();

        Sender client = new Sender(key);
        String ciphertext = client.encrypt(plaintext);

        System.out.println("Ciphertext: " + ciphertext);
```

```java
        try (Socket socket = new Socket("localhost", 5000);
             ObjectOutputStream out = new
ObjectOutputStream(socket.getOutputStream())) {

            out.writeObject(key);
            out.writeObject(ciphertext);
            out.flush();

            System.out.println("Sent to server.");

        } catch (IOException e) {
            e.printStackTrace();
        }
        sc.close();
    }
}
```

Receiver

```java
// Server.java - receiver
import java.io.*;
import java.net.*;

public class Receiver {
    private char[][] matrix;

    public Receiver(String key) {
        buildMatrix(key.toUpperCase().replace('J', 'I'));
    }

    private void buildMatrix(String key) {
        boolean[] used = new boolean[26];
        matrix = new char[5][5];
        int idx = 0;

        for (char c : key.toCharArray()) {
            if (c >= 'A' && c <= 'Z' && !used[c - 'A']) {
                used[c - 'A'] = true;
                matrix[idx / 5][idx % 5] = c;
                idx++;
            }
        }

        for (char c = 'A'; c <= 'Z'; c++) {
            if (c == 'J') continue;
            if (!used[c - 'A']) {
                matrix[idx / 5][idx % 5] = c;
```

```
                idx++;
            }
        }
    }

    private int[] find(char c) {
        for (int i = 0; i < 5; i++) {
            for (int j = 0; j < 5; j++) {
                if (matrix[i][j] == c) {
                    return new int[]{i, j};
                }
            }
        }
        return new int[]{-1, -1};
    }

    public String decrypt(String cipher) {
        StringBuilder plain = new StringBuilder();
        for (int i = 0; i < cipher.length(); i += 2) {
            char a = cipher.charAt(i);
            char b = cipher.charAt(i + 1);
            int[] posA = find(a);
            int[] posB = find(b);

            if (posA[0] == posB[0]) {
                plain.append(matrix[posA[0]][(posA[1] - 1 + 5) % 5])
                     .append(matrix[posB[0]][(posB[1] - 1 + 5) % 5]);
            } else if (posA[1] == posB[1]) {
                plain.append(matrix[(posA[0] - 1 + 5) % 5][posA[1]])
                     .append(matrix[(posB[0] - 1 + 5) % 5][posB[1]]);
            } else {
                plain.append(matrix[posA[0]][posB[1]])
                     .append(matrix[posB[0]][posA[1]]);
            }
        }

        // Remove filler 'X' between identical letters or at end
        StringBuilder result = new StringBuilder();
        for (int i = 0; i < plain.length(); i++) {
            char c = plain.charAt(i);
            if (c == 'X') {
                if (i > 0 && i < plain.length() - 1 && plain.charAt(i - 1) ==
plain.charAt(i + 1))
                    continue;
                if (i == plain.length() - 1)
                    continue;
            }
            result.append(c);
```

```java
        }
        return result.toString();
    }

    public static void main(String[] args) {
        final int PORT = 5000;
        try (ServerSocket serverSocket = new ServerSocket(PORT)) {
            System.out.println("Server ready on port " + PORT);

            try (Socket clientSocket = serverSocket.accept();
                 ObjectInputStream in = new
ObjectInputStream(clientSocket.getInputStream())) {

                String key = (String) in.readObject();
                String ciphertext = (String) in.readObject();

                System.out.println("Received key: " + key);
                System.out.println("Received ciphertext: " + ciphertext);

                Receiver r = new Receiver(key);
                String decrypted = r.decrypt(ciphertext);

                System.out.println("Decrypted plaintext: " + decrypted);

            } catch (Exception e) {
                e.printStackTrace();
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Output [screenshot]

Sender

```
PS E:\aa vit sem6\lab2> javac Sender.java
PS E:\aa vit sem6\lab2> java Sender
Enter key: monarchy
Enter plaintext: balloon
Ciphertext: IBSUPMNA
Sent to server.
```

Receiver

```
PS E:\aa vit sem6\lab2> javac Receiver.java
PS E:\aa vit sem6\lab2> java Receiver
Server ready on port 5000
Received key: monarchy
Received ciphertext: IBSUPMNA
Decrypted plaintext: BALLOON
PS E:\aa vit sem6\lab2>
```

HILL CIPHER

Code

Sender

```
import java.net.*;
import java.io.*;
import java.util.*;

public class sender_hill {
    public static void main(String[] args) throws Exception {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter plaintext (UPPERCASE): ");
        String pt = sc.nextLine().toUpperCase();

        System.out.print("Enter matrix size n: ");
        int n = sc.nextInt();

        if (pt.length() % n != 0) {
            System.out.println("Plaintext length not multiple of matrix
size");
            return;
        }

        int[][] key = new int[n][n];
        System.out.println("Enter key matrix:");
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                key[i][j] = sc.nextInt();

        Socket s = new Socket("localhost", 8000);
        PrintWriter out = new PrintWriter(s.getOutputStream(), true);

        String ct = "";

        for (int i = 0; i < pt.length(); i += n) {
            for (int r = 0; r < n; r++) {
                int sum = 0;
                for (int c = 0; c < n; c++)
                    sum += key[r][c] * (pt.charAt(i + c) - 'A');
                ct += (char) ((sum % 26) + 'A');
            }
        }

        System.out.println("Cipher Text (CT): " + ct);
```

```
        out.println(n);
        out.println(ct);
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++)
                out.print(key[i][j] + " ");
            out.println();
        }

        s.close();
        sc.close();
    }
}
```

Receiver

```
import java.net.*;
import java.io.*;
import java.util.*;

public class receiver_hill {
    public static void main(String[] args) throws Exception {

        System.out.println("Server starting...");
        ServerSocket ss = new ServerSocket(8000);
        System.out.println("Server waiting for client connection...");

        Socket s = ss.accept();
        System.out.println("Client connected.");

        BufferedReader in = new BufferedReader(
                new InputStreamReader(s.getInputStream()));

        int n = Integer.parseInt(in.readLine());

        String ct = in.readLine();
        System.out.println("Cipher Text received: " + ct);

        int[][] key = new int[n][n];
        for (int i = 0; i < n; i++) {
            StringTokenizer st = new StringTokenizer(in.readLine());
            for (int j = 0; j < n; j++) {
                key[i][j] = Integer.parseInt(st.nextToken());
            }
        }


        // Augmented matrix [K | I]
```

```java
        int[][] aug = new int[n][2 * n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++)
                aug[i][j] = key[i][j] % 26;
            aug[i][i + n] = 1;
        }

        // Gauss-Jordan elimination (mod 26)
        for (int i = 0; i < n; i++) {
            int inv = 0;
            for (int x = 1; x < 26; x++)
                if ((aug[i][i] * x) % 26 == 1)
                    inv = x;

            for (int j = 0; j < 2 * n; j++)
                aug[i][j] = (aug[i][j] * inv) % 26;

            for (int k = 0; k < n; k++) {
                if (k != i) {
                    int factor = aug[k][i];
                    for (int j = 0; j < 2 * n; j++)
                        aug[k][j] = (aug[k][j] - factor * aug[i][j]) % 26;
                }
            }
        }

        int[][] invKey = new int[n][n];
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                invKey[i][j] = (aug[i][j + n] + 26) % 26;

        String pt = "";

        for (int i = 0; i < ct.length(); i += n) {
            for (int r = 0; r < n; r++) {
                int sum = 0;
                for (int c = 0; c < n; c++)
                    sum += invKey[r][c] * (ct.charAt(i + c) - 'A');
                pt += (char) ((sum % 26) + 'A');
            }
        }

        System.out.println("Plain Text (PT): " + pt);
        System.out.println("Closing connection.");

        s.close();
        ss.close();
    }
```

```
}
```

Output [screenshot]

Sender

```
PS E:\aa vit sem6\lab2> javac sender_hill.java
PS E:\aa vit sem6\lab2> java sender_hill
Enter plaintext (UPPERCASE): HELP
Enter matrix size n: 2
Enter key matrix:
3
3
2
5
Cipher Text (CT): HIAT
PS E:\aa vit sem6\lab2>
```

Receiver

```
PS E:\aa vit sem6\lab2> javac receiver_hill.java
PS E:\aa vit sem6\lab2> java receiver_hill
Server starting...
Server waiting for client connection...
Client connected.
Cipher Text received: HIAT
Plain Text (PT): HELP
Closing connection.
PS E:\aa vit sem6\lab2>
```